

Assignment 04

JSON to XML Translator using Flex & Yacc

Total Marks: 100 (+10 Bonus)

Assignment Summary

- Build a JSON to XML translator in C using Flex and Yacc/Bison.
- Construct an Abstract Syntax Tree (AST) during parsing.
- Traverse the AST and generate well-formed XML.
- Focus on core compiler concepts rather than extra command-line features.

Important Announcements

Deadline	As per GCR
Submission	RollNumber1-RollNumber2-Section.zip (e.g., 22i1234-22i5678-A.zip)
Language	C, C++, or Java (choose one)

Instructions

- Work in the same pairs as Assignment 3 (groups cannot be changed).
- Must be implemented in C, C++, or Java — choose one language for the entire assignment.
- Code must be generic and reusable.
- Proper indentation, comments, and presentable output required.
- Plagiarism = ZERO marks.
- Submit only .ZIP files on Google Classroom.
- No email submissions accepted.
- Test thoroughly before submission.

Overview

In this assignment, you will build a small translator that reads valid JSON input, parses it using Flex and Yacc/Bison, constructs an Abstract Syntax Tree (AST), and converts the input into well-formed XML. The goal is to practice the main phases of a compiler-style pipeline in a focused and manageable way.

Learning Objectives

- Write a lexer using Flex.
- Write a parser using Yacc/Bison.
- Construct an AST during parsing.
- Traverse the AST after parsing.
- Translate one structured format into another.
- Generate valid XML from a tree structure.
- Report lexical and syntax errors clearly.

Program Requirement

Your program should be named:

```
./json2xml < input.json
```

The program must:

- Read JSON input from standard input.
- Validate the input using Flex and Yacc/Bison.
- Build an AST.
- Convert the AST into XML.
- Print the XML output to standard output.

JSON Features You Must Support

- Objects
- Arrays
- Strings
- Numbers
- Booleans (true, false)
- null

Conversion Rules

Rule 1: Root Wrapper

The final XML output must have exactly one outer wrapper: `<root> ... </root>`.

Rule 2: JSON Object

A JSON object becomes an XML element whose children are generated from its key-value pairs.

Rule 3: Keys to XML Tags

Each JSON key becomes the XML tag name. Test cases will use simple keys that are already valid XML tag names.

Rule 4: Scalar Values

JSON strings, numbers, and booleans become text content inside the corresponding XML tag.

Rule 5: Null

A JSON null becomes an empty XML element.

Rule 6: Arrays

A JSON array becomes a sequence of repeated child elements using the tag `<item>`.

Rule 7: Nested Structures

Objects inside objects, arrays inside objects, and objects inside arrays must preserve nesting.

Implementation Requirements

Lexer

Use Flex to generate tokens for JSON input. Your lexer must recognize:

- Punctuation: { } [] : ,
- String literals
- Number literals
- Keywords: true, false, null

Your lexer should also ignore whitespace correctly. For strings, you only need to support common escaped characters such as `\`, `\\`, `\n`, and `\t`. The lexer should detect basic lexical errors such as invalid characters, invalid escape sequences, and unterminated strings.

Parser

Use Yacc/Bison to parse JSON and build the AST directly. Your grammar must support:

- value
- object
- array
- key-value pair
- scalar values

Do not build a full parse tree. Build a compact AST with meaningful node types.

AST

You must construct an AST during parsing and keep it available until translation is complete. A simple design may use node types such as OBJECT, ARRAY, STRING, NUMBER, BOOL, and NULL. You may use structs, unions, linked lists, or dynamic child arrays. A clean design is preferred over an overly complex one.

XML Generation

After successful parsing, traverse the AST and generate XML. Your XML output must:

- Be well-formed.
- Preserve JSON nesting.
- Use `<item>` for array elements.
- Print null as an empty tag.
- Escape important XML characters in text such as `&`, `<`, and `>`.

Error Reporting

Your program must stop at the first lexical or syntax error and print a clear error message:

```
Error: unexpected token
```

A readable error message is required. More specific wording is appreciated if implemented.

Sample Inputs and Outputs

Example 1: Simple Object

Input	Output
<pre>{ "id": 1, "name": "Ali", "age": 19 }</pre>	<pre><root> <id>1</id> <name>Ali</name> <age>19</age> </root></pre>

Example 2: Array of Scalars

Input	Output
<pre>{ "movie": "Inception", "genres": ["Action", "Sci-Fi", "Thriller"] }</pre>	<pre><root> <movie>Inception</movie> <genres> <item>Action</item> <item>Sci-Fi</item> <item>Thriller</item> </genres> </root></pre>

Example 3: Nested Object

Input	Output
<pre>{ "author": { "uid": "u1", "name": "Sara" }, "published": true }</pre>	<pre><root> <author> <uid>u1</uid> <name>Sara</name> </author> <published>true</published> </root></pre>

Example 4: Null

Input	Output
<pre>{ "editedAt": null }</pre>	<pre><root> <editedAt/> </root></pre>

Submission Instructions

Submit one ZIP file named:

```
Rollno1-Rollno2-Section.zip
```

Your ZIP must contain:

- scanner.l
- parser.y
- All supporting .c and .h files
- Makefile
- README.md
- At least five JSON test files
- Expected XML output for each test

The README must include build instructions, run instructions, a short explanation of the AST structure, and any assumptions or limitations.

Grading Rubric

Area	What We Expect	Marks
Lexer	Correct recognition of JSON tokens, whitespace handling, and basic string/number processing	20
Parser	Correct JSON grammar and successful parsing of valid input	20
AST Construction	Proper AST nodes created during parsing with a clear structure	20
XML Translation	Correct conversion from AST to well-formed XML, including arrays, nesting, scalars, and null	20
Error Handling	Clear lexical/syntax error reporting and clean program exit	10
Code Quality & Submission	Clean code, modularity, naming, README, test files, and correct submission format	10

Core Total = 100 Marks

Bonus Features

Optional Feature	Description	Bonus
Pretty-printed XML	Readable indentation and formatting	+2
AST printing	Clear display of AST structure	+2
Column-based error detail	More informative error messages	+2
Unicode escape support	Handles \u1234 style escapes	+2
Scientific notation support	Accepts numbers with exponent notation	+2

Maximum Bonus = 10 Marks